

Введение в синтаксис языка C#

Сравнение C# с языком Python

Сравнение программы “Hello world” на двух языках

```
print("Hello world!")
```

На Python

```
using System;

namespace HelloWorld
{
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            Console.WriteLine("Hello world!");
        }
    }
}
```

На C#

Характеристика	C#	Python
Скорость выполнения	Выше (JIT-компиляция)	Ниже (интерпретация)
Поддержка ООП	Очень сильная, строгая модель	Гибкая, но менее строгая модель
Использование	Корпоративные приложения, игры (Unity), сервисы под Windows	Скрипты, анализ данных, веб-разработка, машинное обучение
Запуск программы	Компилируется перед выполнением	Выполняется сразу (интерпретация)

Но бывает и исключения (динамический тип данных в C#):

```
static void Main(string[] args)
{
    dynamic x = 4;
    Console.WriteLine(x);
    Console.WriteLine(x.GetType());
    x = "hello";
    Console.WriteLine(x);
    Console.WriteLine(x.GetType());
    x = 3.4;
    Console.WriteLine(x);
    Console.WriteLine(x.GetType());
}
```

Консоль отладки Microsoft V

```
4
System.Int32
hello
System.String
3,4
System.Double
C:\Users\Palto
```

Цикл for (вывод чисел от 1 до 10)

```
for i in range(1, 11):  
    print(i)
```

На Python

```
for (int i = 1; i <= 10; i++)  
{  
    Console.WriteLine(i);  
}
```

На C#

Условия (проверка числа)

```
num = 12
if num < 10:
    print("Меньше 10")
elif 10 <= num <= 15:
    print("От 10 до 15")
else:
    print("Больше 15")
```

На Python

```
int num = 12;
if (num < 10)
{
    Console.WriteLine("Меньше 10");
}
else if (num >= 10 && num <= 15)
{
    Console.WriteLine("От 10 до 15");
}
else
{
    Console.WriteLine("Больше 15");
}
```

На C#

Функция/Метод для суммы двух чисел

```
def sum_numbers(a, b):  
    return a + b  
  
result = sum_numbers(5, 3)  
print(result) # 8
```

На Python

```
static int SumNumbers(int a, int b)  
{  
    return a + b;  
}  
  
int result = SumNumbers(5, 3);  
Console.WriteLine(result); // 8
```

На C#

Простой класс Студент

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        print(f"Привет, меня зовут {self.name}!")

# Использование
student = Student("Алексей", 20)
student.greet() # Привет, меня зовут Алексей!
```

На Python

На C#

```
internal class Program
{
    0 references
    static void Main(string[] args)
    {
        // Использование
        Student student = new("Алексей", 20);
        student.Greet(); // Привет, меня зовут Алексей!
    }
}

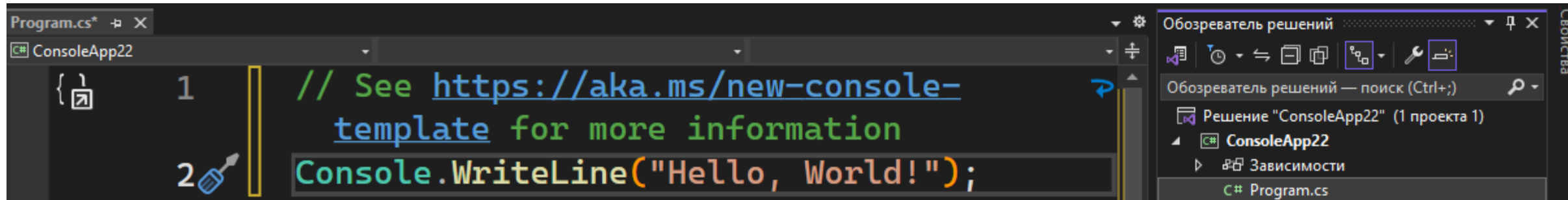
3 references
public class Student
{
    2 references
    public string Name { get; set; }
    1 reference
    public int Age { get; set; }

    1 reference
    public Student(string name, int age)
    {
        Name = name;
        Age = age;
    }

    1 reference
    public void Greet()
    {
        Console.WriteLine($"Привет, меня зовут {Name}!");
    }
}
```

Структура программы

Весь код программы на языке **C#** помещается в файлы с расширением **.cs**. По умолчанию в проекте, который создается в **Visual Studio** (а также при использовании **.NET CLI**) уже есть один файл с кодом **C#** - файл **Program.cs** со следующим содержимым:

A screenshot of the Visual Studio IDE. The main editor window shows the 'Program.cs' file for a project named 'ConsoleApp22'. The code contains two lines: a comment on line 1 pointing to a URL for more information, and a console output statement on line 2. The 'Solution Explorer' on the right shows the project structure with 'ConsoleApp22' and its 'Program.cs' file.

```
1 // See https://aka.ms/new-console-  
  template for more information  
2 Console.WriteLine("Hello, World!");
```

Именно код файла **Program.cs** выполняется по умолчанию, если мы запустим проект на выполнение. Но при необходимости мы также можем добавлять другие файлы с кодом **C#**

Инструкции

Базовым строительным блоком программы являются инструкции (**statement**). Инструкция представляет некоторое действие, например, арифметическую операцию, вызов метода, объявление переменной и присвоение ей значения. В конце каждой инструкции в **C#** ставится точка с запятой (**;**). Данный знак указывает компилятору на конец инструкции.

Например, в проекте консольного приложения, который создается по умолчанию, есть такая строка:

```
Console.WriteLine("Hello, World!");
```

Данная строка представляет вызов метода **Console.WriteLine**, который выводит на консоль строку. В данном случае вызов метода является инструкцией и поэтому завершается точкой с запятой.

Набор инструкций может объединяться в блок кода. Блок кода заключается в фигурные скобки, а инструкции помещаются между открывающей и закрывающей фигурными скобками.

```
{  
    Console.WriteLine("Привет");  
    Console.WriteLine("Добро пожаловать в C#");  
}
```

Регистрозависимость

C# является регистрозависимым языком. Это значит, что в зависимости от регистра символов какие-то определенные названия могут представлять разные классы, методы, переменные и т.д. Например, для вывода на консоль используется метод **WriteLine** - его имя начинается именно с большой буквы: "**WriteLine**". Если мы вместо "**Console.WriteLine**" напомним "**Console.writeline**", то программа не скомпилируется, так как данный метод обязательно должен называться "**WriteLine**", а не "**writeline**" или "**WRITELINE**" или как-то иначе.

Комментарии

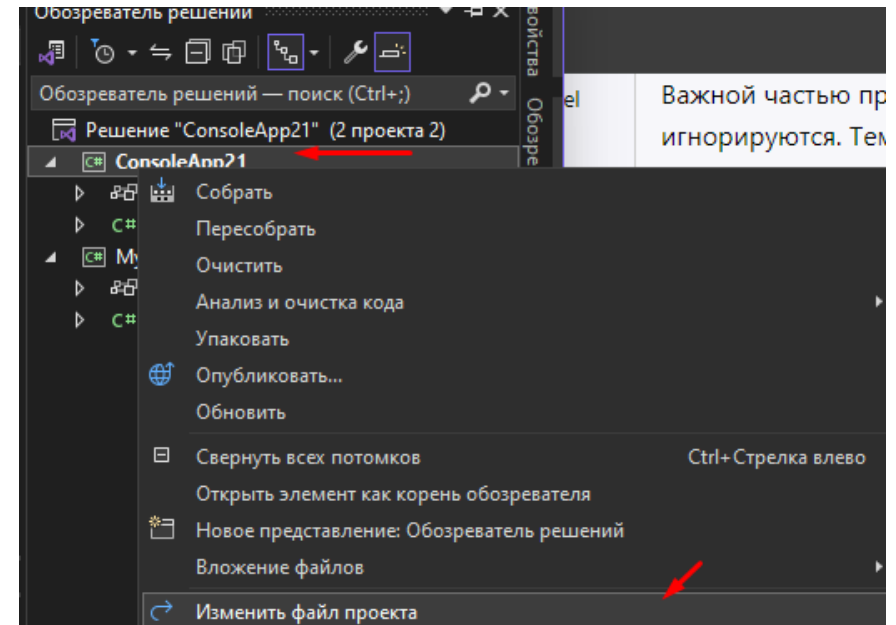
Важной частью программного кода являются комментарии. Они не являются собственно частью программы, при компиляции они игнорируются. Тем не менее комментарии делают код программы более понятным, помогая понять те или иные его части.

Есть два типа комментариев: однострочный и многострочный. Однострочный комментарий размещается на одной строке после двойного слеша `//`. А многострочный комментарий заключается между символами `/* текст комментария */`. Он может размещаться на нескольких строках. Например:

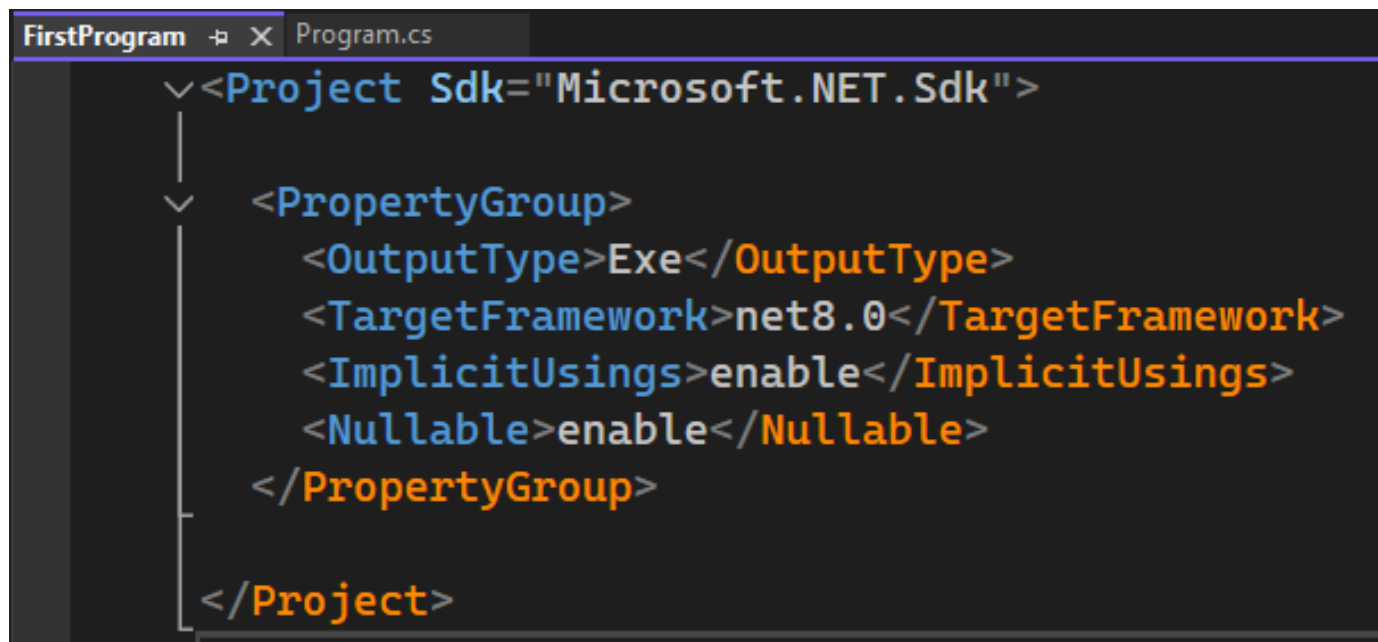
```
/*  
    первая программа на C#,  
    которая выводит приветствие на консоль  
*/  
Console.WriteLine("Привет");  
// Выводим строку "Привет"  
Console.WriteLine("Добро пожаловать в C#");  
// Выводим строку "Добро пожаловать в C#"
```

Файл проекта

В каждом проекте **C#** есть файл, который отвечает за общую конфигурацию проекта. По умолчанию этот файл называется **Название_проекта.csproj**. Итак, откроем данный файл. Для этого либо двойным кликом левой кнопкой мыши нажмем на название проекта, либо нажмем на название проекта правой кнопкой мыши и в появившемся меню выберем пункт **Изменить файл проекта**



После этого Visual Studio откроет нам файл проекта, который будет выглядеть следующим образом:



```
FirstProgram x Program.cs
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

Этот файл в виде кода **xml** определяет конфигурацию проекта и он может содержать множество элементов.

Переменные и константы

Для хранения данных в программе применяются переменные. **Переменная** представляет именованную область памяти, в которой хранится значение определенного типа. Переменная имеет **тип, имя и значение**. Тип определяет, какого рода информацию может хранить переменная.

Перед использованием любую переменную надо определить. Синтаксис определения переменной выглядит следующим образом:

тип имя_переменной;

Требования:

- имя может содержать любые цифры, буквы и символ подчеркивания, при этом первый символ в имени должен быть буквой или символом подчеркивания;
- в имени не должно быть знаков пунктуации и пробелов;
- имя не может быть ключевым словом языка C#. Таких слов не так много, и при работе в Visual Studio среда разработки подсвечивает ключевые слова синим цветом.

При этом следует учитывать, что C# является регистрозависимым языком, поэтому следующие два определения переменных будут представлять две разные переменные:

```
string name;
```

```
string Name;
```

После определения переменной можно присвоить некоторое значение:

```
int age;  
age = 18;
```

Также мы можем сразу при определении присвоить переменной значение. Данный прием называется инициализацией:

```
string name = "Billy";
```


Отличительной чертой переменных является то, что в программе можно многократно менять их значение. Например, создадим небольшую программу, в которой определим переменную, поменяем ее значение и выведем его на консоль:

```
int age = 18;  
Console.WriteLine(age); // 18
```

```
age = 50;  
Console.WriteLine(age); // 50
```

Константы

Особенностью переменных является то, что мы можем изменить их значение в процессе работы программы. Но, кроме того, в C# есть константы. Константа должна быть обязательно инициализирована при определении, и после определения значение константы не может быть изменено.

Константы предназначены для описания таких значений, которые не должны изменяться в программе. Для определения констант используется ключевое слово **const**, которое указывается перед типом константы:

```
1  const int age = 18;
2  Console.WriteLine(age); // 18
3
4  age = 50;
5  Console.WriteLine(age); // !Ошибка - у
    константы нельзя изменить значение
```

Константы

При использовании констант надо помнить, что объявить мы их можем только один раз и что к моменту компиляции они должны быть определены. Так, в следующем случае мы получим ошибку, так как константе не присвоено начальное значение:

```
1  const int age;  
2  Console.WriteLine(age); // ! Ошибка –  
   константа age не инициализирована
```

Литералы

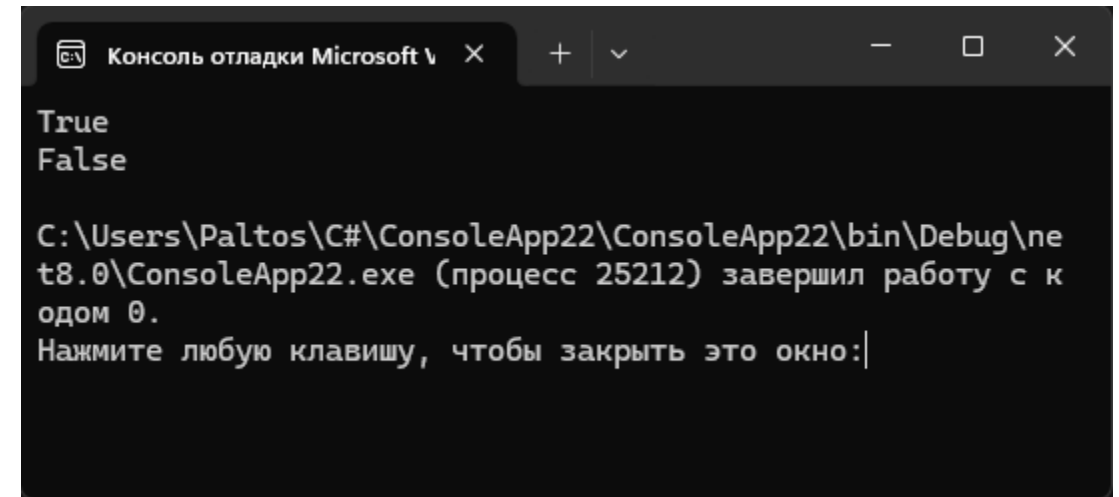
Литералы представляют неизменяемые значения (иногда их еще называют константами). Литералы можно передавать переменным в качестве значения. Литералы бывают логическими, целочисленными, вещественными, символьными и строчными. И отдельный литерал представляет ключевое слово null.

Логические литералы

Есть две логических константы - **true (истина)** и **false (ложь)**:

```
Console.WriteLine(true);
```

```
Console.WriteLine(false);
```



```
Консоль отладки Microsoft Visual Studio
True
False

C:\Users\Paltos\C#\ConsoleApp22\ConsoleApp22\bin\Debug\net8.0\ConsoleApp22.exe (процесс 25212) завершил работу с кодом 0.
Нажмите любую клавишу, чтобы закрыть это окно:
```

Целочисленные литералы

Целочисленные литералы представляют положительные и отрицательные целые числа, например, 1, 2, 3, 4, -7, -109. Целочисленные литералы могут быть выражены в десятичной, шестнадцатеричной и двоичной форме.

```
Console.WriteLine(-14);
```

```
Console.WriteLine(300);
```

Числа в двоичной форме предваряются символами **0b**, после которых идет набор из нулей и единиц:

```
Console.WriteLine(0b11); // 3  
Console.WriteLine(0b1011); // 11
```

Для записи числа в шестнадцатеричной форме применяются символы **0x**, после которых идет набор символов от 0 до 9 и от A до F, которые собственно представляют число:

```
Console.WriteLine(0x0A); // 10  
Console.WriteLine(0xFF); // 255
```


Вещественные литералы

Вещественные литералы представляют дробные числа. Этот тип литералов имеет две формы. Первая форма - вещественные числа с фиксированной запятой, при которой дробную часть отделяется от целой части точкой. Например:

3.14

100.001

-0.38

Также вещественные литералы могут определяться в экспоненциальной форме ME_p , где M — мантисса, E - экспонента, которая фактически означает $"*10^"$ (умножить на десять в степени), а p — порядок.
Например:

```
Console.WriteLine(3.2e3);
```

```
// по сути равно  $3.2 * 10^3 = 3200$ 
```

```
Console.WriteLine(1.2E-1);
```

```
// равно  $1.2 * 10^{-1} = 0.12$ 
```

Символьные литералы

Символьные литералы представляют одиночные символы. Символы заключаются в **одинарные кавычки**.

Символьные литералы бывают нескольких видов. Прежде всего это обычные символы:

```
Console.WriteLine('2');
```

```
Console.WriteLine('A');
```

```
Console.WriteLine('T');
```

Специальную группу представляют управляющие последовательности.

Управляющая последовательность представляет символ, перед которым ставится слеш.

И данная последовательность интерпретируется определенным образом.

Наиболее часто используемые последовательности:

'\n' - перевод строки

'\t' – табуляция

'\\' - слеш

Еще один способ определения символов представляет использования шестнадцатеричных кодов **ASCII**. Для этого в одинарных кавычках указываются символы **'\x'**, после которых идет шестнадцатеричный код символа из таблицы **ASCII**.

```
Console.WriteLine('\x78'); // x
```

```
Console.WriteLine('\x5A'); // z
```

И последний способ определения символьных литералов представляет применение кодов из таблицы символов **Unicode**. Для этого в одинарных кавычках указываются символы **'\u'**, после которых идет шестнадцатеричный код **Unicode**.

```
Console.WriteLine('\u0420'); // Р  
Console.WriteLine('\u0421'); // С
```

Строковые литералы

Строковые литералы представляют строки. Строки заключаются в **двойные кавычки**:

```
Console.WriteLine("hello");
```

```
Console.WriteLine("Как дела?");
```

```
Console.WriteLine("Всё норм");
```

Если внутри строки необходимо вывести двойную кавычку, то такая внутренняя кавычка предваряется **обратным слешем**:

```
Console.WriteLine("Компания \"Microsoft\"");
```

Также в строках можно использовать управляющие последовательности. Например, последовательность **'\n'** осуществляет перевод на новую строку:

```
Console.WriteLine("Привет \nмир");
```


Конкатенация и интерполяция строк

Интерполяция строк призвана упростить форматирование строк. Так, перепишем пример с выводом значений переменных в строке:

```
string name = "Богдан";
```

```
int age = 20;
```

```
// Конкатенация
```

```
Console.WriteLine("Имя: " + name + " Возраст: " + age);
```

```
// Интерполяция строк
```

```
Console.WriteLine($"Имя: {name} Возраст: {age}");
```

null

null представляет ссылку, которая не указывает ни на какой объект. То есть по сути отсутствие значения.

```
string name = null;  
Console.WriteLine(name);
```

Типы данных в C#: примитивные типы и их свойства

Как и во многих языках программирования, в С# есть своя система типов данных, которая используется для создания переменных. Тип данных определяет внутреннее представление данных, множество значений, которые может принимать объект, а также допустимые действия, которые можно применять над объектом.



Тип	Диапазон значений	Размер (байт)
sbyte	-128 ... 127	1
byte	0 ... 255	1
short	-32 768 ... 32 767	2
ushort	0 ... 65 535	2
int	-2 147 483 648 ... 2 147 483 647	4
uint	0 ... 4 294 967 295	4
long	-9 223 372 036 854 775 808 ... 9 223 372 036 854 775 807	8
ulong	0 ... 18 446 744 073 709 551 615	8
float	$\pm 1,5 \cdot 10^{-45} \dots \pm 3,4 \cdot 10^{+38}$, точность – 7 разрядов	4
double	$\pm 5,0 \cdot 10^{-324} \dots \pm 1,7 \cdot 10^{+308}$, точность – 16 разрядов	8
decimal	$\pm 1 \cdot 10^{-28} \dots \pm 7,9 \cdot 10^{+28}$, точность – 28 разрядов	16
bool	true или false	1
char	0 ... 65 535 (код символа)	2

1. Целочисленные типы

`byte = System.Byte`

хранит целое число от 0 до 255

1 byte = 8 bit

`byte bit1 = 1;`

`byte bit2 = 100;`

`sbyte = System.SByte`

хранит целое число -128 до 127

1 byte = 8 bit

`sbyte bit3 = -101;`

`sbyte bit4 = 102;`

`short = System.Int16`

хранит целое число

от -32768 до 32767

2 byte = 16 bit

`short n1 = 1;`

`short n2 = 102;`

`ushort = System.UInt16`

хранит целое число от 0 до 65535

2 byte = 16 bit

`ushort n3 = 1;`

`ushort n4 = 102;`

1. Целочисленные типы

`int = System.Int32`

хранит целое число от

`-2147483648` до `2147483647`

`4 byte = 32 bit`

`int integer1 = 10;`

`int integer2 = 0b101;` // бинарная
форма `b = 5`

`uint = System.UInt32`

хранит целое число от

`0` до `4294967295`

`4 byte = 32 bit`

`uint integer3 = 10;`

`uint integer4 = 0b101;`

`long = System.Int64`

хранит целое число от

`-9 223 372 036 854 775 808` до

`9 223 372 036 854 775 807`

`8 byte = 64 bit`

`long number1 = -10;`

`long number2 = 0b101;`

`ulong = System.UInt64`

хранит целое число от `0` до

`18 446 744 073 709 551 615`

`8 byte = 64 bit`

`ulong number3 = 10;`

`ulong number4 = 0b101;`

2. Булевы(Логические) типы

```
bool = System.Boolean
```

//хранит значение true или false (логические литералы)

```
bool alive = true;
```

```
bool isDead = false;
```


3. Числа с плавающей точкой

float = **System.Single**

//хранит число с плавающей точкой от

// $-3.4 \cdot 10^{38}$ до $3.4 \cdot 10^{38}$

//4 byte = 32 bit

//~6 - 8 знаков, после запятой

float floating1 = **3.14F**;

float floating2 = **30.6f**;

double = **System.Double**

//хранит число с плавающей точкой от $\pm 5.0 \cdot 10^{-324}$ до

$\pm 1.7 \cdot 10^{308}$

//8 byte = 64 bit

//~15 - 16 знаков, после запятой

double yNumber1 = **3.14**;

double yNumber2 = **36.6**;

4. Десятичное дробное число

`decimal = System.Decimal`

`//от  $\pm 1.0 \cdot 10^{-28}$  до  $\pm 7.9228 \cdot 10^{28}$`

`//16 byte = 128 bit`

`//~25 - 28 знаков, после запятой`

`decimal dNumber1 = 4.5m;`

`decimal dNumber2 = 23.56M;`

5. СИМВОЛЫ

`char = System.Char`

//хранит одиночный символ в кодировке Unicode-16

`2 byte = 16 bit`

`char s = 'A';`

`char d = ' ';`

6. Строки

```
string = System.String  
// хранит набор символов Unicode
```

```
string hello = "Hello";  
string word = "world";
```

7. Object

object = System.Object

// может хранить значение любого типа данных

4 byte = 32 bit(32-bit system)

8 byte = 64 bit(64-bit system)

object ob1 = 22;

object ob2 = 3.14;

object ob3 = "hello code";

Особенность расчётов вещественных чисел

```
double a = 0.1;  
double b = 0.2;  
double c = a + b;  
double d = 0.3;  
Console.WriteLine(c == d); // False
```

```
decimal x = 0.1m;  
decimal y = 0.2m;  
decimal z = x + y;  
decimal e = 0.3m;  
Console.WriteLine(z == e); // True
```

Использование системных типов

Выше при перечислении всех базовых типов данных для каждого упоминался системный тип. Потому что название встроенного типа по сути представляет собой сокращенное обозначение системного типа. Например, следующие переменные будут эквивалентны по типу:

```
int metr1 = 1;
```

```
System.Int32 metr2 = 1;
```

Неявная типизация

Ранее мы явным образом указывали тип переменных, например, `int x`; . И компилятор при запуске уже знал, что `x` хранит целочисленное значение.

Однако мы можем использовать и модель неявной типизации:

```
var hello = "Hell to World";  
var c = 20;
```


Для неявной типизации вместо названия типа данных используется ключевое слово **var**. Затем уже при компиляции компилятор сам выводит тип данных исходя из присвоенного значения. Так как по умолчанию все целочисленные значения рассматриваются как значения типа **int**, то поэтому в итоге переменная **c** будет иметь тип **int**. Аналогично переменной **hello** присваивается строка, поэтому эта переменная будет иметь тип **string**.

Эти переменные подобны обычным, однако они имеют некоторые ограничения.

Во-первых, мы не можем сначала объявить неявно типизируемую переменную, а затем инициализировать:

```
// этот код работает
```

```
int number;
```

```
number = 300;
```

```
// этот код не работает
```

```
var number2;
```

```
number2= 300;
```

Во-вторых, мы не можем указать в качестве значения неявно типизируемой переменной **null**:

```
// этот код не работает. Так как значение  
null, то компилятор не сможет вывести тип  
данных.
```

```
var c = null;
```

Консольный ввод-вывод

Консольный вывод

Для вывода информации на консоль мы уже использовали встроенный метод `Console.WriteLine`. То есть, если мы хотим вывести некоторую информацию на консоль, то нам надо передать ее в метод `Console.WriteLine`:

```
string hello = "Привет мир";  
Console.WriteLine(hello);  
Console.WriteLine("Я изучаю C#");  
Console.WriteLine("Пока мир...");  
Console.WriteLine(36.6);
```

Нередко возникает необходимость вывести на консоль в одной строке значения сразу нескольких переменных. В этом случае мы можем использовать прием, который называется **интерполяцией**:

```
string name = "Billy";
```

```
int age = 48;
```

```
double height = 1.85;
```

```
Console.WriteLine($"Имя: {name} Возраст: {age} Рост:  
{height}м");
```

Есть другой способ вывода на консоль сразу нескольких значений (плейсхолдеры):

```
string name = "Billy";
```

```
int age = 48;
```

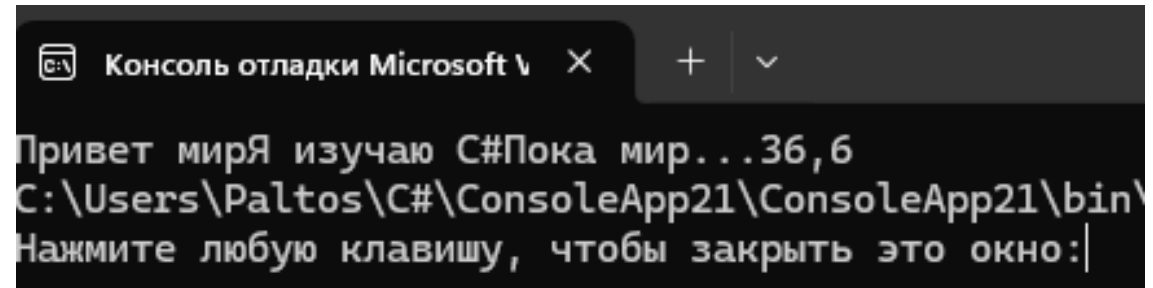
```
double height = 1.85;
```

```
Console.WriteLine("Имя: {0} Возраст: {1} Рост: {2}м",  
name, age, height);
```

Console.Write

Кроме `Console.WriteLine()` можно также использовать метод `Console.Write()`, он работает точно так же за тем исключением, что не добавляет переход на следующую строку, то есть последующий консольный вывод будет выводиться на той же строке.

```
string hello = "Привет мир";  
Console.Write(hello);  
Console.Write("Я изучаю C#");  
Console.Write("Пока мир...");  
Console.Write(36.6);
```

A screenshot of the Microsoft Visual Studio Console window. The title bar shows 'Консоль отладки Microsoft V' with a close button. The console output displays the concatenated strings and the number: 'Привет мирЯ изучаю C#Пока мир...36,6'. Below the output, the file path 'C:\Users\PaltoS\C#\ConsoleApp21\ConsoleApp21\bin\' is visible, followed by the instruction 'Нажмите любую клавишу, чтобы закрыть это окно:|'.

```
Консоль отладки Microsoft V X + v  
Привет мирЯ изучаю C#Пока мир...36,6  
C:\Users\PaltoS\C#\ConsoleApp21\ConsoleApp21\bin\  
Нажмите любую клавишу, чтобы закрыть это окно:|
```


Консольный ввод

Кроме вывода информации на консоль мы можем получать информацию с консоли. Для этого предназначен метод **Console.ReadLine()**. Он позволяет получить введенную строку.

```
Console.Write("Введите свое имя: ");  
string? name = Console.ReadLine();  
Console.WriteLine($"Привет {name}");
```

Здесь **string** указывает, что переменная может хранить значения типа **string**, то есть строки. А знак вопроса **?** указывает, что переменная также может хранить значение **null**, то есть по сути не иметь никакого значения.

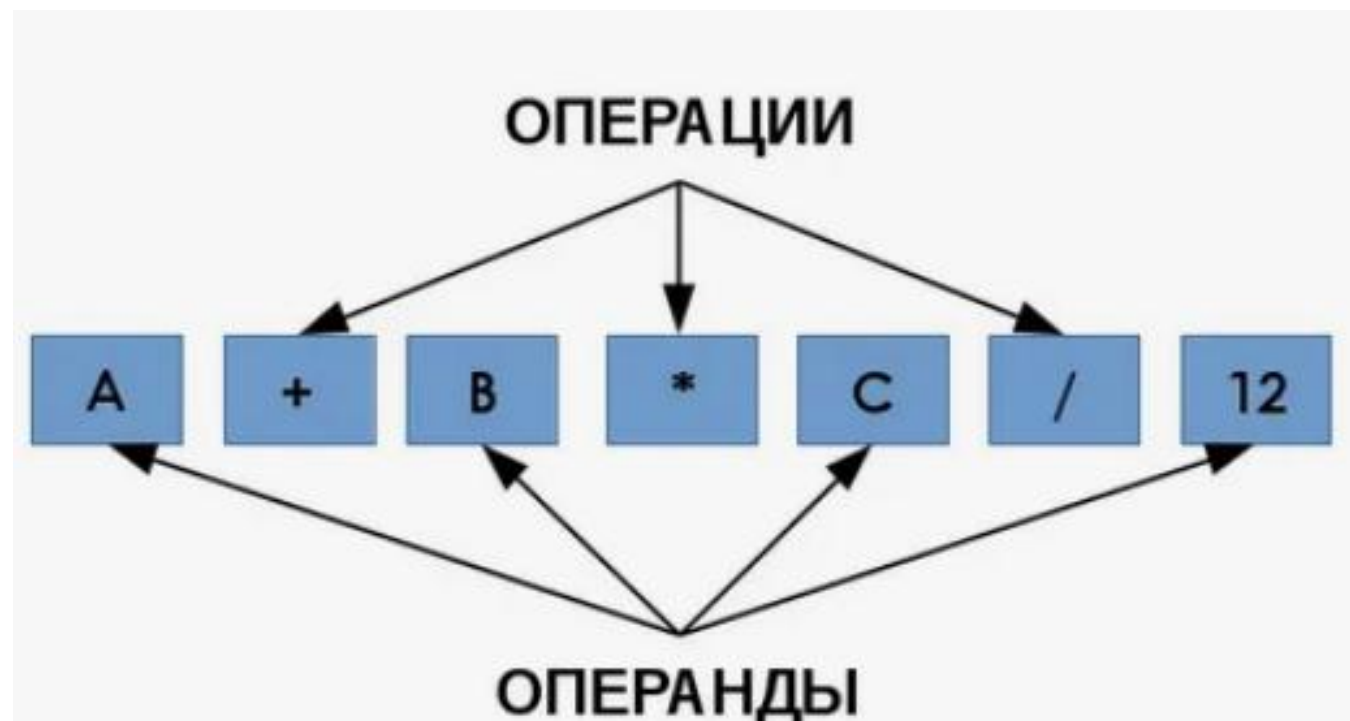
Однако, может возникнуть вопрос, как нам быть, если, допустим, мы хотим ввести возраст в переменную типа **int** или другую информацию в переменные типа **double** или **decimal**? По умолчанию платформа **.NET** предоставляет ряд методов, которые позволяют преобразовать различные значения к типам **int**, **double** и т.д. Некоторые из этих методов:

- **Convert.ToInt32()** (преобразует к типу **int**)
- **Convert.ToDouble()** (преобразует к типу **double**)
- **Convert.ToDecimal()** (преобразует к типу **decimal**)

```
Console.Write("Введите имя: ");
string? name = Console.ReadLine();
Console.Write("Введите возраст: ");
int age = Convert.ToInt32(Console.ReadLine());
Console.Write("Введите рост: ");
double height = Convert.ToDouble(Console.ReadLine());
Console.Write("Введите размер зарплаты: ");
decimal salary = Convert.ToDecimal(Console.ReadLine());
Console.WriteLine($"Имя: {name} Возраст: {age} Рост: {height}м Зарплата: {salary}$");
```

Арифметические операции

В С# используется большинство операций, которые применяются и в других языках программирования. Операции представляют определенные действия над операндами - участниками операции.



В качестве операнда может выступать переменная или какое-либо значение (например, число). Операции бывают унарными (выполняются над одним операндом), бинарными - над двумя операндами и тернарными - выполняются над тремя операндами. Рассмотрим все виды операций.

Бинарные арифметические операции

// Операция сложения двух чисел:

```
int x1 = 100;
```

```
int y1 = x1 + 40; // 140
```

```
Console.WriteLine(y1);
```

// Операция вычитания двух чисел:

```
int x2 = 100;
```

```
int y2 = x2 - 40; // 60
```

```
Console.WriteLine(y2);
```

// Операция умножения двух чисел:

```
int x3 = 20;
```

```
int y3 = x3 * 5; // 100
```

```
Console.WriteLine(y3);
```

// Операция деления двух чисел:

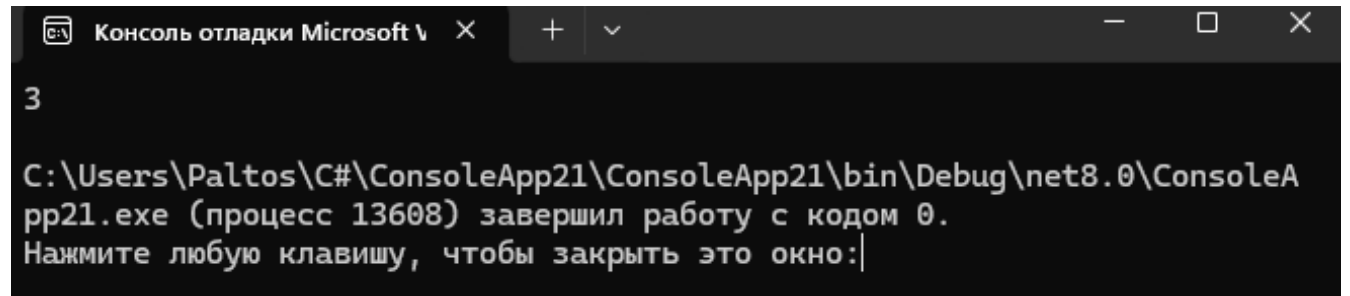
```
int x4 = 20;
```

```
int y4 = x4 / 5; // 4
```

```
Console.WriteLine(y4);
```

Что мы получим в следующем примере?

```
int x = 10;  
int y = x / 3;  
Console.WriteLine(y);
```



The screenshot shows a Windows console window titled "Консоль отладки Microsoft v". The output of the program is the number "3". Below the output, the console displays the message: "C:\Users\Paltos\C#\ConsoleApp21\ConsoleApp21\bin\Debug\net8.0\ConsoleApp21.exe (процесс 13608) завершил работу с кодом 0. Нажмите любую клавишу, чтобы закрыть это окно:".

При делении стоит учитывать, что если оба операнда представляют целые числа, то результат также будет округляться до целого числа.

Что мы получим в следующих примерах?

```
double a = 10;
```

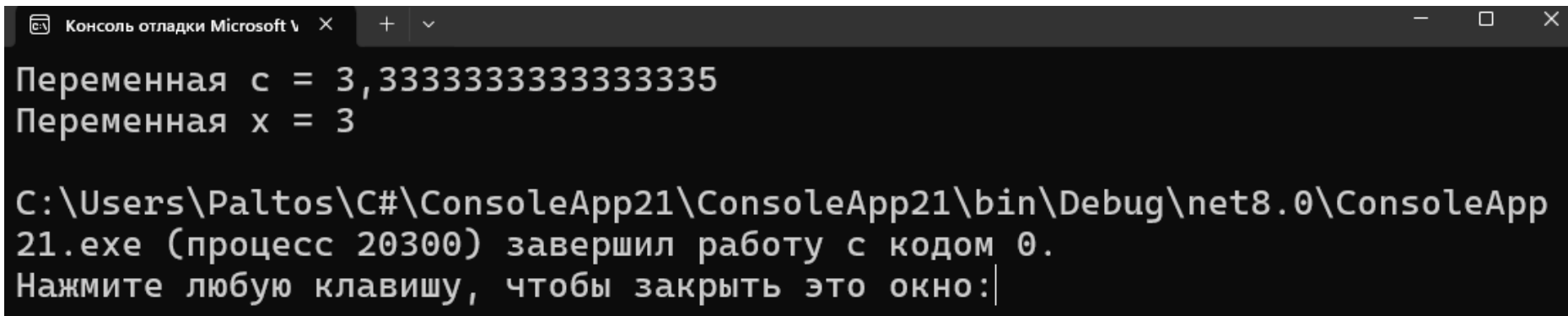
```
double b = 3;
```

```
double c = a / b;
```

```
Console.WriteLine($"Переменная c = {c}");
```

```
double x = 10 / 3;
```

```
Console.WriteLine($"Переменная x = {x}");
```



```
Консоль отладки Microsoft Visual Studio
Переменная c = 3,3333333333333335
Переменная x = 3

C:\Users\Paltos\C#\ConsoleApp21\ConsoleApp21\bin\Debug\net8.0\ConsoleApp21.exe (процесс 20300) завершил работу с кодом 0.
Нажмите любую клавишу, чтобы закрыть это окно:
```

Несмотря на то, что результат операции в итоге помещается в переменную типа **double**, которая позволяет сохранить дробную часть, но в самой операции участвуют два литерала, которые по умолчанию рассматриваются как объекты **int**, то есть целые числа, и результат то же будет целочисленный.

Для выхода из этой ситуации необходимо определять литералы или переменные, участвующие в операции, именно как типы **double** или **float**:

```
double x = 10.0 / 3.0;
```

```
Console.WriteLine($"Переменная x = {x}");
```

Бинарные арифметические операции

// Операция получение остатка от целочисленного деления двух чисел

```
int x = 10;
```

```
int y = x % 3; //результат равен 1
```

$$\begin{array}{r|l} 10 & 3 \\ \hline 9 & \\ \hline 1 & \end{array} \quad \begin{array}{r|l} 10 & 4 \\ \hline 8 & \\ \hline 2 & \end{array}$$

Унарные операции. Операция инкремента ++

// Инкремент бывает префиксным: ++x - сначала значение переменной x увеличивается на 1, а потом ее значение возвращается в качестве результата операции.

```
int x1 = 4;
```

```
int y1 = ++x1;
```

```
Console.WriteLine($"x1 = {x1}, y1 = {y1}");
```

```
// x1 = 5, y1 = 5
```

// И также существует постфиксный инкремент: x++ - сначала значение переменной x возвращается в качестве результата операции, а затем к нему прибавляется 1.

```
int x2 = 4;
```

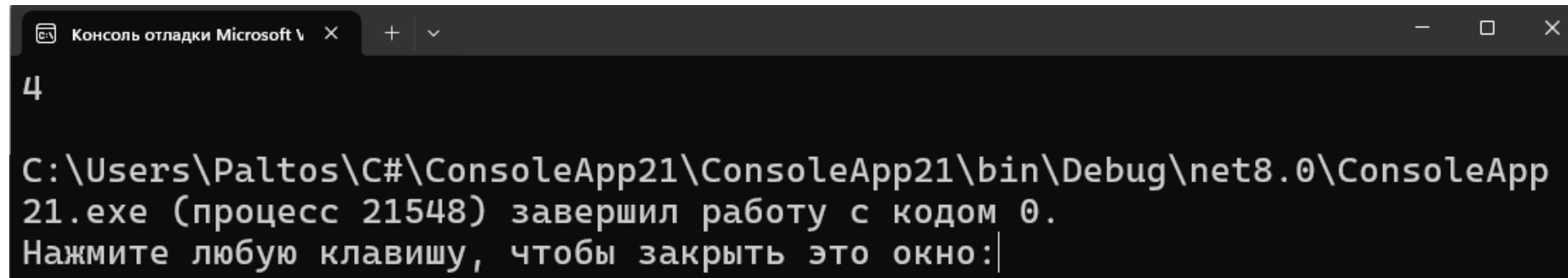
```
int y2 = x2++;
```

```
Console.WriteLine($"x2 = {x2}, y2 = {y2}");
```

```
// x2 = 5, y2 = 4
```

Что мы получим в следующем примере?

```
int x = 1;  
Console.WriteLine(x++ + ++x);
```



The screenshot shows a Windows Command Prompt window titled "Консоль отладки Microsoft \". The output of the program is displayed as follows:

```
4  
  
C:\Users\PaltoS\C#\ConsoleApp21\ConsoleApp21\bin\Debug\net8.0\ConsoleApp  
21.exe (процесс 21548) завершил работу с кодом 0.  
Нажмите любую клавишу, чтобы закрыть это окно:
```

1. `x++` возвращает 1, `x` становится 2.
2. `++x` увеличивает `x` до 3 и возвращает 3.
3. Сложение 1 и 3 дает 4.

Унарные операции. Операция декремента --

//Декремент бывает префиксным: --x
сначала значение переменной x
уменьшается на 1, а потом ее значение
возвращается в качестве результата
операции.

```
int x1 = 4;
```

```
int y1 = --x1;
```

```
Console.WriteLine($"x1 = {x1}, y1 = {y1}");
```

```
// x1 = 3, y1 = 3
```

//И также существует постфиксный
декремент: x-- сначала значение
переменной x возвращается в качестве
результата операции, а затем из него
вычитается 1.

```
int x2 = 4;
```

```
int y2 = x2--;
```

```
Console.WriteLine($"x2 = {x2}, y2 = {y2}");
```

```
// x2 = 3, y2 = 4
```

При выполнении сразу нескольких арифметических операций следует учитывать порядок их выполнения.

Приоритет операций от наивысшего к низшему:

1. Инкремент, декремент
2. Умножение, деление, получение остатка
3. Сложение, вычитание

Для изменения порядка следования операций применяются скобки.

Ассоциативность операторов

Операции умножения и деления имеют один и тот же приоритет, но какой тогда результат будет в выражении:

```
int x = 10 / 5 * 2;
```

```
// или мы 10 делим на 5 = 2, и умножаем на 2 = 4.
```

```
// или мы 5 умножаем на 2 = 10, и затем 10 делим на 10 = 1.
```

```
Console.WriteLine(x);
```

Все арифметические операторы являются левоассоциативными, то есть
выполняются слева направо

Операции присваивания

Операции присвоения устанавливают значение. В операциях присвоения участвуют два операнда, причем левый операнд может представлять только модифицируемое именованное выражение, например, переменную.

Как и во многих других языках программирования, в C# имеется базовая операция присваивания `=`, которая присваивает значение правого операнда левому операнду:

```
int x = 10;
```



Здесь переменной `x` присваивается число `10`. Переменная `x` представляет левый операнд, которому присваивается значение правого операнда, то есть числа `10`.

Также можно выполнять множественно присвоение сразу нескольким переменным одновременно:

```
int a, b, c;  
a = b = c = 50;
```

Стоит отметить, что операции присвоения имеют низкий приоритет. И вначале будет вычисляться значение правого операнда и только потом будет идти присвоение этого значения левому операнду. Например:

```
int a, b, c;  
a = b = c = 10 * 3 / 5; // 6
```

Сначала будет вычисляться значение $10 * 3 / 5$, а затем полученное значение будет присвоено переменным.

Кроме базовой операции присвоения в C# есть еще ряд операций:

+=: присваивание после сложения. Присваивает левому операнду сумму левого и правого операндов:

выражение $A += B$ равнозначно выражению $A = A + B$

```
int x = 4;  
x += 5; // 9
```

-=: присваивание после вычитания. Присваивает левому операнду разность левого и правого операндов: $A -= B$ эквивалентно $A = A - B$

```
int x = 4;  
x -= 1; // 3
```

***=**: присваивание после умножения. Присваивает левому операнду произведение левого и правого операндов: $A *= B$ эквивалентно $A = A * B$

```
int x = 4;  
x *= 2; // 8
```

/=: присваивание после деления. Присваивает левому операнду частное левого и правого операндов:

$A /= B$ эквивалентно $A = A / B$

```
int x = 12;  
x /= 3; // 4
```

%=: присваивание после деления по модулю. Присваивает левому операнду остаток от целочисленного деления левого операнда на правый:

A %= B эквивалентно **A = A % B**

```
int x = 12;
```

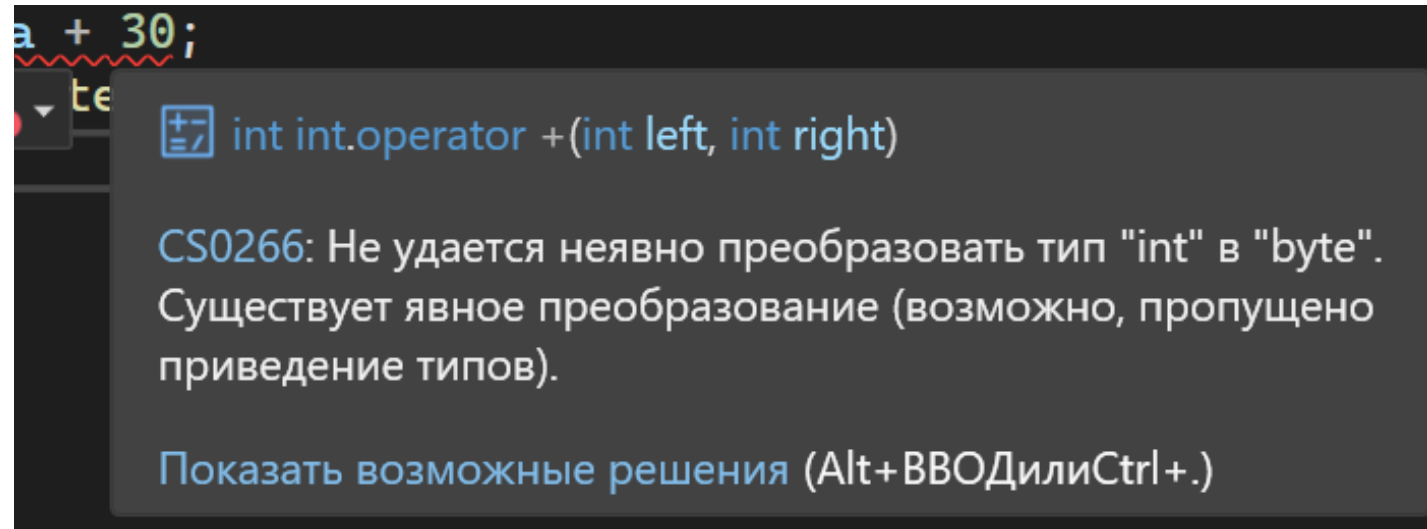
```
x %= 3; // 0
```

```
int y = 12;
```

```
y %= 5; // 2
```


Преобразование базовых типов данных

```
int a = 10;  
byte b = a + 30;
```



Операции сложения и вычитания возвращают значение типа **int**, если в операции участвуют целочисленные типы данных с разрядностью меньше или равно **int** (то есть типы **byte**, **short**, **int**). Поэтому результатом операции **a + 30** будет объект, который имеет длину в памяти 4 байта. Затем этот объект мы пытаемся присвоить переменной **b**, которая имеет тип **byte** и в памяти занимает **1 байт**.

И чтобы выйти из этой ситуации, необходимо применить операцию преобразования типов. **Операция преобразования** типов предполагает указание в скобках того типа, к которому надо преобразовать значение:

(тип в который преобразовываем) значение;

```
int a = 10;  
byte b = (byte)(a + 30);
```

Сужающие и расширяющие преобразования

Преобразования могут быть сужающие (narrowing) и расширяющие (widening). Расширяющие преобразования расширяют размер объекта в памяти. Например:

```
byte a = 4;           // 0000100
ushort b = a;         // 0000000000000100
```

В данном случае переменной типа ushort присваивается значение типа **byte**. Тип **byte** занимает **1 байт (8 бит)**, и значение переменной **a** в двоичном виде можно представить как:

00000100

Значение типа **ushort** занимает **2 байта (16 бит)**. И при присвоении переменной **b** значение переменной **a** расширяется до **2 байт**

000000000000000100

То есть значение, которое занимает **8 бит**, **расширяется** до **16 бит**.

Сужающие преобразования, наоборот, сужают значение до типа меньшей разрядности.

```
ushort a = 4;  
byte b = (byte) a;
```

Здесь переменной **b**, которая занимает **8 бит**, присваивается значение **ushort**, которое занимает **16 бит**. То есть из **0000000000000000100** получаем **00000100**. Таким образом, значение сужается с **16 бит (2 байт)** до **8 бит (1 байт)**.

Неявные преобразования

В случае с расширяющими преобразованиями компилятор за нас выполнял все преобразования данных, то есть преобразования были неявными (implicit conversion). Такие преобразования не вызывают каких-то затруднений. Тем не менее стоит сказать пару слов об общей механике подобных преобразований.

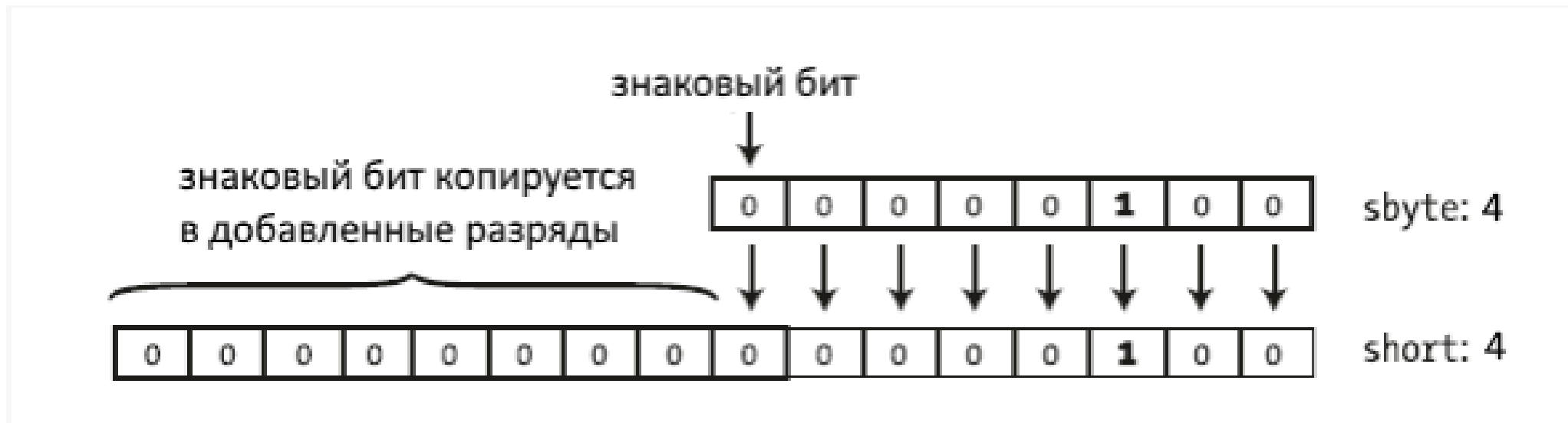
Если производится преобразование от беззнакового типа меньшей разрядности к беззнаковому типу большей разрядности, то добавляются дополнительные биты, которые имеют значение 0. Это называется дополнение нулями или zero extension.

```
byte a = 4;           // 0000100
ushort b = a;         // 0000000000000100
```

Если производится преобразование к знаковому типу, то битовое представление дополняется нулями, если число положительное, и единицами, если число отрицательное. Последний разряд числа содержит знаковый бит - 0 для положительных и 1 для отрицательных чисел. При расширении в добавленные разряды копируется знаковый бит.

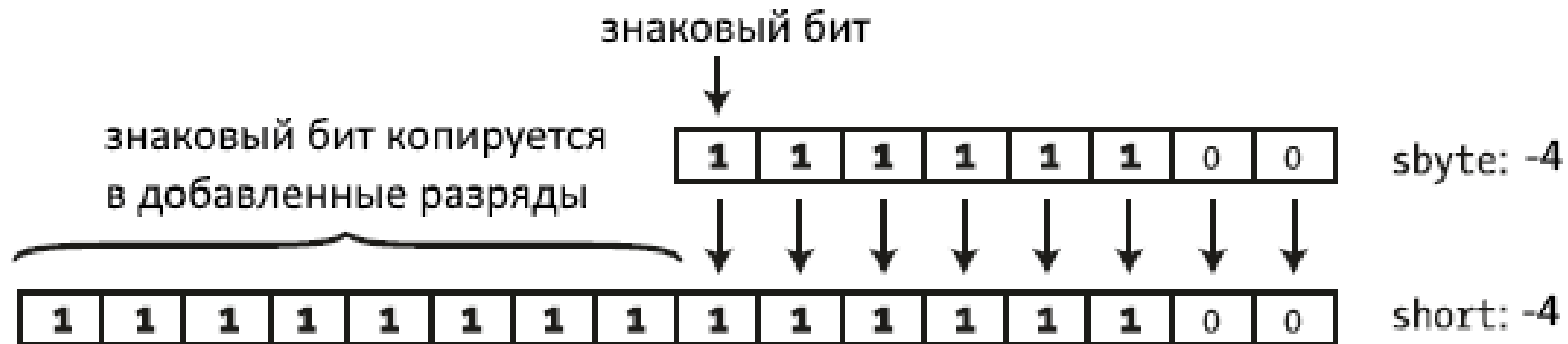
Рассмотрим преобразование положительного числа:

```
sbyte a = 4;           // 0000100
short b = a;           // 0000000000000100
```



Преобразование отрицательного числа:

```
sbyte a = -4;           // 1111100
short b = a;            // 1111111111111100
```



Явные преобразования

При явных преобразованиях (explicit conversion) мы сами должны применить операцию преобразования (операция `()`). Суть операции преобразования типов состоит в том, что перед значением указывается в скобках тип, к которому надо привести данное значение:

```
int a = 4;  
int b = 6;  
byte c = (byte)(a+b);
```

Расширяющие преобразования от типа с меньшей разрядностью к типу с большей разрядностью компилятор проводит неявно. Это могут быть следующие цепочки преобразований:

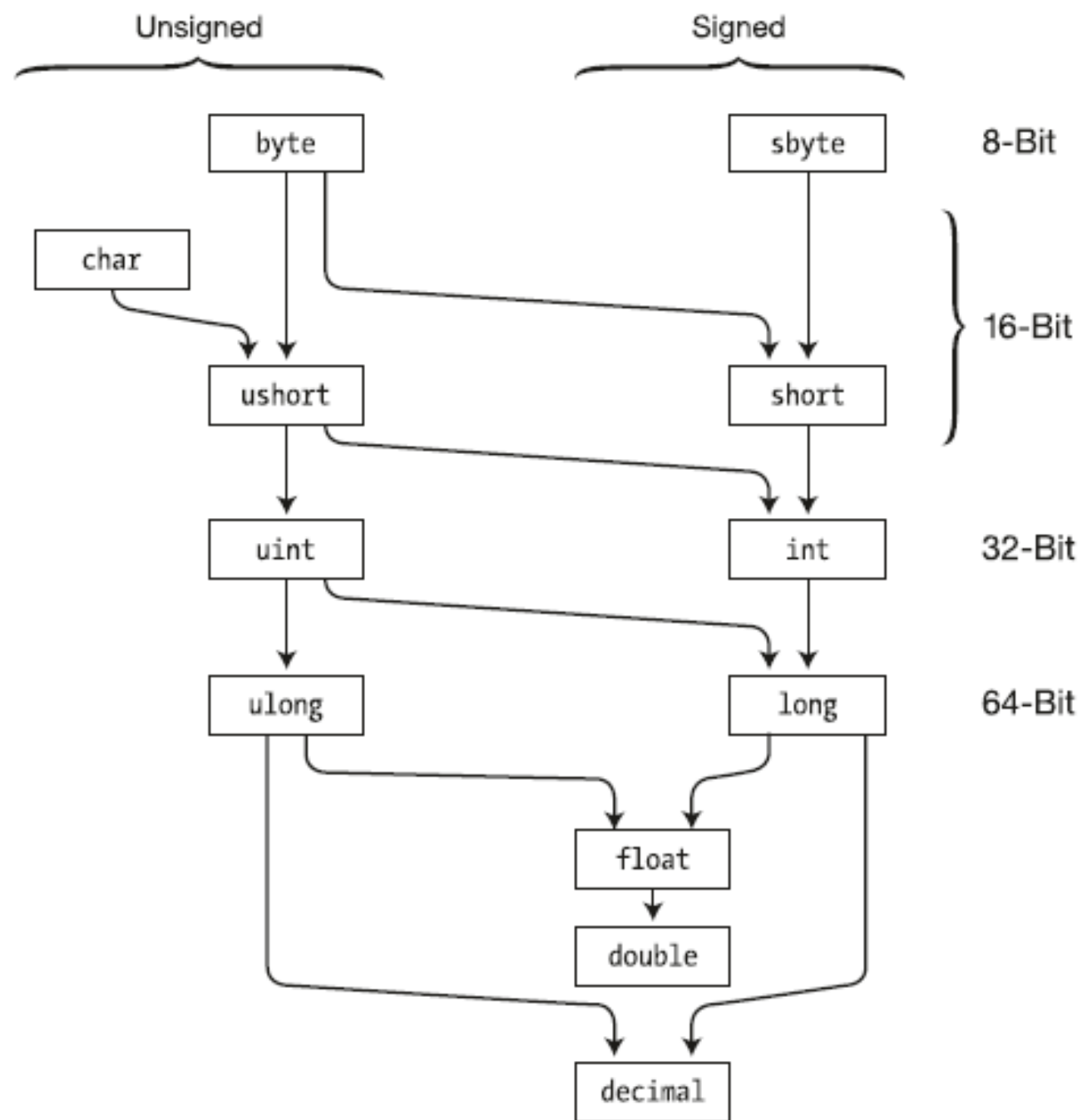
byte -> short -> int -> long -> decimal

int -> double

short -> float -> double

char -> int

Тип	В какие типы безопасно преобразуется
byte	short, ushort, int, uint, long, ulong, float, double, decimal
sbyte	short, int, long, float, double, decimal
short	int, long, float, double, decimal
ushort	int, uint, long, ulong, float, double, decimal
int	long, float, double, decimal
uint	long, ulong, float, double, decimal
long	float, double, decimal
ulong	float, double, decimal
float	double
char	ushort, int, uint, long, ulong, float, double, decimal



Потеря точности данных

```
int a = 250;  
int b = 10;  
byte c = (byte)(a+b); // 4
```

Результатом будет число **4**, так число **260** не попадает в допустимый диапазон для типа **byte**, и старшие биты будут усекаться. Поэтому при преобразованиях надо это учитывать. И мы в данном случае можем либо взять такие числа *a* и *b*, которые в сумме дадут число не больше **255**, либо мы можем выбрать вместо **byte** другой тип данных, например, **int**.

Условные выражения

Отдельный набор операций представляет условные выражения. Такие операции возвращают логическое значение, то есть значение типа **bool: true**, если выражение истинно, и **false**, если выражение ложно. К подобным операциям относятся операции сравнения и логические операции.

Операции сравнения

В операциях сравнения сравниваются два операнда и возвращается значение типа **bool** - **true**, если выражение верно, и **false**, если выражение неверно.

==

//Сравнивает два операнда на равенство. Если они равны, то операция возвращает true, если не равны, то возвращается false:

```
int a = 25;
```

```
int b = 17;
```

```
Console.WriteLine(a == b); // false
```

!=

//Сравнивает два операнда и возвращает true, если операнды не равны, и false, если они равны.

```
int a = 45;
```

```
int b = 13;
```

```
Console.WriteLine(a != b); // true
```

```
Console.WriteLine(a != 45); // false
```

<

//Операция "меньше чем". Возвращает true, если первый операнд меньше второго, и false, если первый операнд больше второго:

```
int a = 45;
```

```
int b = 13;
```

```
Console.WriteLine(a < b); // false
```

>

//Операция "больше чем". Сравнивает два операнда и возвращает true, если первый операнд больше второго, иначе возвращает false:

```
int a = 45;
```

```
int b = 13;
```

```
Console.WriteLine(a > b); // true
```

<=

//Операция "меньше или равно". Сравнивает два операнда и возвращает true, если первый операнд меньше или равен второму. Иначе возвращает false.

```
int a = 45;
```

```
int b = 13;
```

```
Console.WriteLine(a <= b); // false
```

>=

//Операция "больше или равно". Сравнивает два операнда и возвращает true, если первый операнд больше или равен второму, иначе возвращается false:

```
int a = 45;
```

```
int b = 13;
```

```
Console.WriteLine(a >= b); // true
```

Логические операции

Также в C# определены логические операторы, которые также возвращают значение типа **bool**. В качестве операндов они принимают значения типа **bool**. Как правило, применяются к отношениям и объединяют несколько операций сравнения.

|

//Операция логического сложения или логическое ИЛИ. Возвращает true, если хотя бы один из операндов возвращает true.

bool x1 = (5 > 6) | (4 < 6); // 5 > 6 - false,

4 < 6 - true, поэтому возвращается true

bool x2 = (5 > 6) | (4 > 6); // 5 > 6 - false,

4 > 6 - false, поэтому возвращается false

&

//Операция логического умножения или логическое И. Возвращает true, если оба операнда одновременно равны true.

bool x1 = (5 > 6) & (4 < 6); // 5 > 6 - false, 4 < 6 - true, поэтому возвращается false

bool x2 = (5 < 6) & (4 < 6); // 5 < 6 - true, 4 < 6 - true, поэтому возвращается true

||

//Операция логического сложения. Возвращает true, если хотя бы один из операндов возвращает true.

```
bool x1 = (5 > 6) || (4 < 6); // 5 > 6 - false,  
4 < 6 - true, поэтому возвращается true  
bool x2 = (5 > 6) || (4 > 6); // 5 > 6 - false,  
4 > 6 - false, поэтому возвращается false
```

&&

//Операция логического умножения. Возвращает true, если оба операнда одновременно равны true.

```
bool x1 = (5 > 6) && (4 < 6); // 5 > 6 - false,
```

4 < 6 - true, поэтому возвращается false

```
bool x2 = (5 < 6) && (4 < 6); // 5 < 6 - true,
```

4 < 6 - true, поэтому возвращается true

!

//Операция логического отрицания. Производится над одним операндом и возвращает true, если операнд равен false. Если операнд равен true, то операция возвращает false:

```
bool a = true;
```

```
bool b = !a; // false
```

^

//Операция исключающего ИЛИ. Возвращает true, если либо первый, либо второй операнд (но не одновременно) равны true, иначе возвращает false

bool x5 = (5 > 6) ^ (4 < 6); // 5 > 6 - false,
4 < 6 - true, поэтому возвращается true

bool x6 = (50 > 6) ^ (4 / 2 < 3); // 50 > 6 - true, 4/2 < 3 - true, поэтому
возвращается false

Здесь у нас две пары операций `|` и `||` (а также `&` и `&&`) выполняют похожие действия, однако же они не равнозначны.

В выражении `z=x|y`; будут вычисляться оба значения - `x` и `y`.

В выражении же `z=x||y`; сначала будет вычисляться значение `x`, и если оно равно `true`, то вычисление значения `y` уже смысла не имеет, так как у нас в любом случае уже `z` будет равно `true`. Значение `y` будет вычисляться только в том случае, если `x` равно `false`

Конструкция if..else и тернарная операция

Условные конструкции - один из базовых компонентов многих языков программирования, которые направляют работу программы по одному из путей в зависимости от определенных условий. Одной из таких конструкций в языке программирования **C#** является конструкция **if..else**

Конструкция **if/else** проверяет истинность некоторого условия и в зависимости от результатов проверки выполняет определенный код. Ее простейшая форма состоит из блока **if**:

```
if(условие) {  
    выполняемые инструкции  
}
```

```
int number1 = 25;  
int number2 = 14;  
if(number1 > number2) {  
    Console.WriteLine($"Число {number1} больше числа  
{number2}");  
}
```

Если блок **if** содержит одну инструкцию, то мы можем его сократить, убрав фигурные скобки:

```
int number1 = 25;  
int number2 = 14;  
if(number1 > number2)  
    Console.WriteLine($"Число {number1} больше числа {number2}");
```

Также мы можем соединить сразу несколько условий, используя логические операторы:

```
int number1 = 25;  
int number2 = 14;  
if(number1 > number2 && number1==25)  
    Console.WriteLine($"Число {number1} больше числа {number2}");
```

Выражение else

Блок **else** выполняется, если условие после **if** ложно, то есть равно **false**.

```
int num1 = 8;  
int num2 = 6;  
if(num1 > num2)  
    Console.WriteLine($"Число {num1} больше числа {num2}");  
else  
    Console.WriteLine($"Число {num1} меньше числа {num2}");
```

else if

Используя конструкцию **else if**, мы можем обрабатывать дополнительные условия.

```
int num1 = 8;  
int num2 = 6;  
if(num1 > num2)  
    Console.WriteLine($"Число {num1} больше числа {num2}");  
else if (num1 < num2)  
    Console.WriteLine($"Число {num1} меньше числа {num2}");  
else  
    Console.WriteLine("Число num1 равно числу num2");
```

Тернарная операция

Тернарная операция позволяет проверить некоторое условие и в зависимости от его истинности выполнить некоторые действия. Она имеет следующий синтаксис:

[первый операнд - условие] ?

[второй операнд] : [третий операнд]

Здесь сразу три операнда. В зависимости от условия тернарная операция возвращает второй или третий операнд: если условие равно **true**, то возвращается второй операнд; если условие равно **false**, то третий.

```
int x=3;  
int y=2;  
int z = x < y ? (x+y) : (x-y);  
Console.WriteLine(z); // 1
```

Здесь первый операнд (то есть условие) представляет выражение $x < y$. Если оно равно **true**, то возвращается второй операнд - $(x+y)$, то есть результат операции сложения. Если условие равно false, то возвращается третий операнд - $(x-y)$.

Результат тернарной операции (то есть второй или третий операнд в зависимости от условия) присваивается переменной **z**.

Циклы: for, foreach, while, do...while

Циклы являются управляющими конструкциями, позволяя в зависимости от определенных условий выполнять некоторое действие множество раз. В C# имеются следующие виды циклов:

- for
- foreach
- while
- do...while

Цикл for

Цикл **for** имеет следующее формальное определение:

```
for ([действия_до_выполнения_цикла]; [условие];  
[действия_после_выполнения])  
{  
    // действия  
}
```

```
for (int i = 1; i < 10; i++) {  
    Console.WriteLine(i);  
}
```

```
1  
2  
3  
4  
5  
6  
7  
8  
9
```

Сокращенная запись:

```
for (int i = 1; i < 10; i++)  
    Console.WriteLine(i);
```

Нам необязательно указывать все условия при объявлении цикла. Например, мы можем написать так:

```
int i = 1;  
for (; ; ) {  
    Console.WriteLine($"i = {i}");  
    i++;  
}
```

Также стоит отметить, что можно определять несколько переменных в объявлении цикла:

```
for (int i = 1, j = 1; i < 10; i++, j++)  
    Console.WriteLine($"{i * j}");
```

Цикл do..while

В цикле **do** сначала выполняется код цикла, а потом происходит проверка условия в инструкции **while**. И пока это условие истинно, цикл повторяется.

```
do {  
    действия цикла  
}  
while (условие)
```

```
int i = 10;  
do {  
    Console.WriteLine(i);  
    i--;  
}  
while (i > 0);
```

Цикл while

В отличие от цикла **do** цикл **while** сразу проверяет истинность некоторого условия, и если условие истинно, то код цикла выполняется:

```
while (условие) {  
    действия цикла  
}
```

```
int i = 10;  
while (i > 0) {  
    Console.WriteLine(i);  
    i--;  
}
```


Но важно отметить, что цикл **do** гарантирует хотя бы однократное выполнение действий, даже если условие в инструкции **while** не будет истинно:

```
int i = -1;
do {
    Console.WriteLine(i); // -1
    i--;
}
while (i > 0);
```

```
int j = -1;
while (j > 0) {
    Console.WriteLine(j); // вывод
    отсутствует
    j--;
}
```

Цикл foreach

Цикл **foreach** предназначен для перебора набора или коллекции элементов. Его общее определение

```
foreach(тип_данных переменная in коллекция) {  
    // действия цикла  
}
```

Цикл foreach

Например, возьмем строку. Строка по сути - это коллекция символов. И **.NET** позволяет перебрать все элементы строки - ее символы с помощью цикла **foreach**

```
foreach(char i in "Hello world") {  
    Console.WriteLine(i);  
}
```

Здесь цикл **foreach** пробегается по всем символам строки **"Hello world"** и каждый символ помещает в символьную переменную **i**. В блоке цикла значение переменной **i** выводится на консоль.

H
e
l
l
o

w
o
r
l
d

Операторы `continue` и `break`

Иногда возникает ситуация, когда требуется выйти из цикла, не дожидаясь его завершения. В этом случае мы можем воспользоваться оператором `break`

```
for (int i = 0; i < 9; i++) {  
    if (i == 5) break;  
    Console.WriteLine(i);  
}
```

Хотя в условии цикла сказано, что цикл будет выполняться, пока счетчик `i` не достигнет значения `9`, в реальности цикл сработает **5 раз**. Так как при достижении счетчиком `i` значения `5`, сработает оператор `break`, и цикл завершится.

0
1
2
3
4

Теперь поставим себе другую задачу. А что если мы хотим, чтобы при проверке цикл не завершался, а просто пропускал текущую итерацию. Для этого мы можем воспользоваться оператором **continue**

```
for (int i = 0; i < 9; i++) {  
    if (i == 5) continue;  
    Console.WriteLine(i);  
}
```

```
0  
1  
2  
3  
4  
6  
7  
8
```

Вложенные циклы

Одни циклы могут быть вложенными в другие. Например:

```
for (int i = 1; i < 10; i++) {  
    for (int j = 1; j < 10; j++) {  
        Console.Write($"{i * j} \t");  
    }  
    Console.WriteLine();  
}
```

Конструкция switch

Конструкция `switch/case` оценивает некоторое выражение и сравнивает его значение с набором значений. И при совпадении значений выполняет определенный код.

Конструкция **switch** имеет следующее формальное определение:

```
switch (выражение)
{
    case значение1:
        код, выполняемый если выражение имеет значение1
        break;
    case значение2:
        код, выполняемый если выражение имеет значение2
        break;
    //.....
    case значениеN:
        код, выполняемый если выражение имеет значениеN
        break;
    default:
        код, выполняемый если выражение не имеет ни одно из выше указанных
значений
        break;
}
```

```
string game = "Dota 2";  
switch (game) {  
    case "Dota 2":  
        Console.WriteLine("Ваша игра - Dota 2");  
        break;  
    case "CS":  
        Console.WriteLine("Ваша игра - Counter-Strike");  
        break;  
    case "Minecraft":  
        Console.WriteLine("Ваша игра - Minecraft");  
        break;  
}
```

Если значение переменной **game** не совпадает ни с каким значением после операторов **case**, то ни один из блоков case не выполняется. Однако если даже в этом случае нам все равно надо выполнить какие-нибудь действия, то мы можем добавить в конструкцию **switch** необязательный блок **default**.

```
string game = "Skyrim";  
switch (game) {  
    case "Dota 2":  
        Console.WriteLine("Ваша игра - Dota 2");  
        break;  
    case "CS":  
        Console.WriteLine("Ваша игра - Counter-Strike");  
        break;  
    case "Minecraft":  
        Console.WriteLine("Ваша игра - Minecraft");  
        break;  
    default:  
        Console.WriteLine("Игра не найдена");  
        break;  
}
```

Однако если мы хотим, чтобы, наоборот, после выполнения текущего блока **case** выполнялся другой блок **case**, то мы можем использовать вместо **break** оператор **goto case**

```
case "Minecraft":
```

```
    Console.WriteLine("Ваша игра - Minecraft");
```

```
    goto case "Dota 2";
```

```
default:
```

```
    Console.WriteLine("Игра не найдена");
```

```
    break;
```